VIZJA University, Warsaw
Network Issues in Mobile Systems (2026–2027)

Project Documentation — Part 1: Conceptual Design

Milestone M1 — Analysis & Design

Kantor Mobile — Mobile Currency Exchange System

Project A · Laboratory Group 7

Authors

Duygu Toplu · Hasan Mert Yilmaz

Supervisor

Marcin Kacprowicz

November 2026

https://github.com/hasanmertyilmaz/nims-currency-exchange

Table of Contents

1. General Information **4**

1.1. Status of this document 4

2. Project Description **5**

2.1. Project Objective 5

2.2. Project Scope 5

2.3. Key design decisions 6

2.4. Main user journeys 6

2.5. Initial risk list 7

3. System Requirements **8**

3.1. Functional Requirements 8

3.2. Non-Functional Requirements 9

3.3. Read operations versus data-changing operations 10

4. UML Diagrams **11**

4.1. Use Case Diagram 11

4.2. Class Diagram 12

4.3. Client-side state of a data-changing operation 16

4.4. Requirements traceability 16

5. Database Design **18**

5.1. Entity Relationship Diagram 18

5.2. Entities, keys and data types 18

5.3. Relationships 21

5.4. Rationale for data types 21

5.5. Consistency rules enforced by the database 21

5.6. Data stored on the mobile device 22

6. System Architecture **23**

6.1. Components and interconnections 23

6.2. Technology stack 24

6.3. Data flows 24

DF-1 — Normal read: current rates 24

DF-2 — Data-changing action: buy currency 25

DF-3 — Recovery of an interrupted action 27

6.4. API contract 27

6.5. Network-failure handling design 29

6.5.1. Connectivity detection 29

6.5.2. Timeout policy 30

6.5.3. Retry policy 30

6.5.4. Stable operation identity and duplicate protection 30

6.5.5. Recovery when a server response is lost 31

6.5.6. Offline mode and cache 31

6.5.7. Reconnection and synchronisation 31

6.5.8. Consistency under concurrent requests 32

6.5.9. Failure of the external data source 32

6.6. Test environment and fault injection (planned) 32
6.7. Network-failure scenario and test matrix 32
7. Project Implementation Plan 34
7.1. Team responsibilities 35
7.2. Repository and working practice 35
8. Conclusions and Possible Extensions 36
9. Sources 37
Appendix A. M1 submission contents 38

1. General Information

Table 1. General project information

Field	Value
Project Name	Kantor Mobile — Mobile Currency Exchange System (Project A)
Project Authors	Duygu Toplu, Hasan Mert Yilmaz — Laboratory Group 7
Field of Study	Computer Science and Artificial Intelligence
Year / Semester	Final year, winter semester 2026/2027
Course	Network Issues in Mobile Systems (2026–2027)
Supervisor	Marcin Kacprowicz
Milestone	M1 — Analysis & Design (teaching-week checkpoint: after Week 4)
Submission Date	November 2026 (date of the Moodle upload)
Git repository	https://github.com/hasanmertyilmaz/nims-currency-exchange (private; the supervisor is given read access). Revision for M1: tag `m1`; the commit hash is stated in the Moodle submission.

1.1. Status of this document

This report is the conceptual design required for milestone M1. It specifies requirements, models, the database, the service interface and the network-failure behaviour that the team will implement in weeks 5–15. **No application code has been written yet.** All timeouts, limits and performance figures below are design targets; they will be implemented and measured at M2 and M3, and the observed results will be added to the network-failure test matrix (Section 6.7). Where the course handbook leaves a value open (timeout, retry count, cache age, token format), the value chosen here is a team decision and its rationale is given.

2. Project Description

2.1. Project Objective

The aim of the project is to create a mobile application that enables **virtual currency exchange**: a registered user can browse current and historical exchange rates published by the National Bank of Poland (NBP), add simulated funds to a PLN wallet, and buy or sell foreign currencies with this virtual money, keeping separate balances per currency and a complete transaction history. No real money is involved at any point; every funding operation is labelled as simulated in the interface.

Because the application runs on a mobile device that communicates over an unreliable wireless link, the central engineering objective of the course is **reliable behaviour under poor connectivity**. The system must detect when the server cannot be reached, never wait indefinitely for a response, show useful cached information offline together with its age, recover after reconnection, and — most importantly for a financial domain — guarantee that a repeated or interrupted buy, sell or deposit request never executes twice and is never reported as successful without confirmation from the server.

User value: students and other users can practise currency exchange with real NBP prices without financial risk, and can rely on balances and history being correct even when the mobile connection is lost during an operation.

2.2. Project Scope

The system consists of four modules. The web service and its database hold the authoritative state; the mobile application only presents it and keeps a local, non-authoritative cache.

Table 2. System modules and their roles

Module	Role
Mobile application (React Native, Expo)	User interface for registration, login, rates, history charts, wallets, deposit, buy/sell and transaction history. Detects connectivity, applies timeouts and the read-retry policy, caches read-only data with its retrieval time, blocks data-changing actions while offline, keeps a journal of unconfirmed operations and reconciles them after reconnection.
Web service (Java 21, Spring Boot 3, REST/JSON)	Business logic and validation, authentication (JWT) and per-user authorisation, NBP integration and rate storage, atomic execution of deposits and exchanges, duplicate-request protection through operation identities, and a reconciliation endpoint that reports the authoritative outcome of an operation.
Database (PostgreSQL 16)	Persistent, authoritative storage of users, currencies, wallets and balances, NBP rate tables, transactions and stored outcomes of data-changing operations. Enforces consistency with keys, unique and check constraints and row-level locks.
External service (NBP Web API)	Source of current and historical exchange rates (table C: bid and ask prices). Accessed only by the web service, never directly by the mobile application.

In scope: registration and login; current and historical rates for PLN against USD, EUR, GBP and CHF; simulated PLN deposits; buying and selling these currencies against PLN; multiple wallets per user; transaction history; offline state, cache, timeouts, retries, reconnection and synchronisation; duplicate protection and recovery from lost responses; a reproducible network-failure test procedure.

Out of scope: real payments or withdrawals, exchanges between two foreign currencies, administrator functions, push notifications and queued offline modifications (see Section 6.5.6 and Section 8).

Planned mobile screens: Login, Register, Rates (home), Rate history (chart and list), Wallets, Deposit (simulated funds), Exchange (buy/sell with confirmation), Transaction history, Pending-operation

banner/dialog and a global connectivity banner.

2.3. Key design decisions

Table 3. Summary of key design decisions

Topic	Decision	Rationale
Rate source	NBP table C (bid/ask) for trading; supported currencies USD, EUR, GBP, CHF	Table C publishes buying and selling prices, so a buy and a sell use different, realistic prices. Table A only provides mid rates.
Authoritative state	Web service + PostgreSQL decide every outcome; the device cache is never authoritative	Handbook: the service validates data-changing actions and records the authoritative outcome.
Offline modifications	**Blocked**, not queued	Prices change and balances must be checked by the server. Blocking is simpler to implement, simpler to test and cannot claim success without confirmation.
Duplicate protection	Client-generated UUID v4 operation id sent as `Idempotency-Key`; outcome stored in `operation_record`	One stable identity per user action; a resend has exactly one effect and receives the original response.
Lost response	Operation marked UNCERTAIN, then resolved with `GET /operations/{id}`	Distinguishes “server committed but response lost” from “server never received the request”.
Money representation	`NUMERIC(19,4)` in the database, `BigDecimal` in Java, decimal strings in JSON	Binary floating point cannot represent decimal currency amounts exactly.
Authentication	Stateless JWT access token (HS256, 24 h), passwords hashed with BCrypt	Simple to implement and to test from the mobile client; no server-side session store.

2.4. Main user journeys

Two normal journeys and two journeys interrupted by connectivity loss guide the design and the final demonstration.

1. **J1 — Normal: first purchase.** A new user registers, logs in, adds 1,000.00 PLN of simulated funds, opens Rates, chooses EUR, reviews the price for 100.00 EUR at the current ask rate and confirms. The server executes the purchase atomically; the app shows the new PLN and EUR balances and the transaction appears in the history.
2. **J2 — Normal: analysis and sale.** A user opens the 30-day EUR history chart, decides to sell 50.00 EUR, confirms at the bid rate, and sees the PLN credit and the updated history.
3. **J3 — Interrupted: connection lost after confirming a sale.** The radio link drops right after the sell request is sent. The app does not know whether the sale happened, marks it “not confirmed yet”, blocks new trades and, after reconnection, asks the server for the outcome of that operation id. It then shows either the completed sale (exactly once) or that the sale was not executed, with the option to retry using the same operation id.
4. **J4 — Interrupted: no connection while browsing.** The user opens the app on the underground without coverage. The app opens from local storage and shows the last rates, balances and history, each labelled with the time of retrieval; Deposit, Buy and Sell are disabled with an explanation. When the connection returns, the app synchronises automatically and the labels change to current data.

2.5. Initial risk list

Table 4. Technical and project risks

Risk	Consequence	Mitigation in the design
Stale data shown as current	User decides on an outdated rate or balance	Every cached view carries its retrieval time; NBP table number and effective date are shown; trading uses the server's latest table and rejects a changed table (RATE_CHANGED).
Uncertain outcome after a lost response	App reports failure although the server executed the operation, or the user repeats it	Operation journal with UNCERTAIN state, reconciliation endpoint, same operation id on resend.
Duplicate execution	Balance debited twice	Idempotency key, `operation_record` primary key, unique (user, operation id) on transactions, per-user row locks.
Concurrent requests from two devices	Negative balance or lost update	Single database transaction with `SELECT … FOR UPDATE` on wallets and `CHECK (balance >= 0)`.
NBP API unavailable or changed	No current rates	Rates stored in the database; service keeps serving the last table with a stale flag; trading stops only if the table is older than 5 days.
Token expiry while offline	User cannot reconcile after reconnecting	Journal persists across logins; reconciliation runs after the next successful login.
Two-person team, limited time	Features or tests incomplete at M3/M4	Small scope (four currencies, no admin role), weekly checkpoints aligned with the handbook, network-failure tests planned from M1.

3. System Requirements

3.1. Functional Requirements

Table 5. Functional requirements

ID	Function Name	Description	Priority
F1	User registration	A guest can create an account with e-mail, password (8–72 characters) and display name. E-mail must be unique. A PLN wallet with balance 0.00 is created in the same database transaction.	High
F2	User login	A registered user logs in with e-mail and password and receives an access token valid for 24 hours; the session survives an app restart.	High
F3	Logout	The user logs out; the token and the user's cached data are removed from the device (unresolved operations are kept, see F17).	Medium
F4	View current rates	The app shows bid and ask prices of USD, EUR, GBP and CHF against PLN from the latest NBP table C, with the table number, its effective date and the time the data was retrieved.	High
F5	View historical rates	The user selects a currency and a period (7, 30 or 90 days; at most 93 days, the NBP query limit) and sees a chart and a list of bid/ask prices per business day.	High
F6	Add simulated funds	The user adds virtual PLN (1.00–100,000.00 per operation) to the PLN wallet. The screen and the history label the operation "simulated".	High
F7	Buy currency	The user enters an amount of foreign currency, reviews the price (amount × ask, rounded to 0.01 PLN) and the NBP table used, and confirms. The service debits PLN and credits the foreign wallet atomically; a foreign wallet is created on first purchase.	High
F8	Sell currency	The user sells an amount of foreign currency, reviews the proceeds (amount × bid, rounded to 0.01 PLN) and confirms. The service debits the foreign wallet and credits PLN atomically.	High
F9	Multiple wallets and balances	The user sees all wallets with balances and the time of the last change.	High
F10	Transaction history	The user sees deposits, buys and sells, newest first, 20 per page, with amounts, currencies, rate, NBP table and time.	High
F11	Service-side validation and authorisation	Every request is validated by the service (amount format and limits, supported currency, balance). A user can read and change only their own wallets, transactions and operations, independently of what the UI hides.	High
F12	NBP integration	The service downloads table C from the NBP API on a schedule, stores it, serves it to clients and fetches historical ranges on demand. Clients never call NBP directly.	High
F13	Offline state indicator	The app detects that the service cannot be reached and shows a persistent, understandable offline banner.	High
F14	Offline cached information	Without connection the app shows the last retrieved rates, histories, balances and transactions, each with a freshness label (time of retrieval).	High
F15	Blocking of offline modifications	While offline, or while an operation is unconfirmed, Deposit, Buy and Sell are disabled and the reason is shown. No change is queued and no success is shown.	High

ID	Function Name	Description	Priority
F16	Duplicate-request protection	Each confirmed deposit, buy or sell carries a stable operation id. Any repetition with the same id has exactly one effect and returns the original result.	High
F17	Uncertain-outcome resolution	If a data-changing request times out or its response is lost, the app marks it “not confirmed”, asks the service for the outcome after reconnection and shows the authoritative result (completed, rejected or not executed with an explicit retry).	High
F18	Synchronisation after reconnection	When connectivity returns the app resolves unconfirmed operations first, then refreshes balances, history and rates, replacing cached data with server data, and shows progress and completion.	High
F19	Rate-change confirmation	If the NBP table changed between review and confirmation the service rejects the request (RATE_CHANGED) and the app shows the new price for a fresh confirmation.	Medium

3.2. Non-Functional Requirements

The template’s example “response time ≤ 2 s” is replaced by measurable targets that the team can test. Network policy values are justified in Section 6.5.

Table 6. Non-functional requirements

ID	Name	Description	Category
N1	Service response time	Measured at the service on the development environment with 10 concurrent simulated users: p95 ≤ 500 ms for reads served from the database, p95 ≤ 1,000 ms for deposit/buy/sell. Justification: about 1 s keeps the user’s flow uninterrupted [11]; the mobile link adds its own latency on top.	Performance
N2	Client timeouts and feedback	Read requests time out after 10 s, data-changing requests after 15 s. A loading indicator appears after 300 ms; after 3 s the message “Still working…” is shown. No request waits indefinitely.	Reliability / Usability
N3	Read retry policy	Only GET requests are retried automatically: at most 3 attempts, delays 1 s and 2 s with ±20 % random jitter, only after a network error, timeout or HTTP 502/503/504. Retries stop immediately when the device goes offline.	Reliability
N4	Exactly-one effect of data-changing requests	Deposit, buy and sell are never re-sent automatically with a new identity. Requests with the same operation id have exactly one effect. Stored outcomes are kept for the lifetime of the account.	Reliability / Consistency
N5	Offline detection	The offline banner appears within 3 s of losing connectivity (platform network events) or immediately after a request fails with a network error.	Reliability
N6	Local cache	Current rates, the last viewed history per currency, wallets and the first page of transactions are stored on the device, survive an app restart, are tied to the user id and are deleted on logout. Every cached view shows its retrieval time; rates older than 24 h are marked “outdated”.	Availability
N7	Synchronisation	Synchronisation starts within 2 s after connectivity is stable (debounce against flapping links) and follows the order in Section 6.5.7. Confirmed server data always overwrites local data; local data never overwrites server data.	Consistency

ID	Name	Description	Category
N8	Transactional consistency	Every deposit, buy and sell is one database transaction; balances can never become negative (database check constraint); for every wallet, balance = sum of credits − sum of debits in its transactions.	Data integrity
N9	Security of credentials and transport	Passwords stored only as BCrypt hashes (cost ≥ 10); access token in the device's secure storage (Expo SecureStore); HTTPS for every non-local deployment; no secrets in the Git repository.	Security
N10	Authorisation	Every protected endpoint derives the user from the token; resources of other users are not found (404). Unauthorised requests are rejected by the service, not only hidden in the UI.	Security
N11	Monetary precision	Amounts stored as NUMERIC(19,4) and processed as BigDecimal; inputs have at most 2 decimal places; PLN results rounded HALF_UP to 0.01. No floating-point arithmetic for money on client or server.	Correctness
N12	Understandable states	Each screen distinguishes loading, success, validation error, business rejection, offline and "not confirmed yet". Validation errors and network errors use different messages.	Usability
N13	Rate freshness	The service refreshes table C at start-up and every 30 min on business days 07:30–16:00 Europe/Warsaw. If the newest stored table is older than 5 calendar days, trading is rejected (RATES_UNAVAILABLE).	Data quality
N14	Portability	The app runs on Android (emulator, API level 34, and one physical device); iOS through Expo Go where available.	Portability
N15	Reproducibility and maintainability	The README describes configuration without secrets, start-up with Docker Compose for the database, database migrations (Flyway) and the steps to reproduce the network-failure demonstration.	Maintainability

3.3. Read operations versus data-changing operations

The handbook requires a clear separation between reads, which may be retried and served from cache, and actions that need authoritative confirmation.

Table 7. Classification of service operations

Operation	Kind	Automatic retry	When offline
Rates, history, wallets, transactions, profile (GET)	Read	Yes (N3)	Served from cache with freshness label
Operation status `GET /operations/{id}`	Read (reconciliation)	Yes (N3)	Deferred until reconnection
Login	Read-like (no state change)	No — user repeats	Blocked with message
Registration	Data-changing	No — a repeat after a lost response returns EMAIL_TAKEN and the app offers login	Blocked
Deposit, buy, sell	Data-changing, financial	No automatic resend; manual resend only with the same operation id after reconciliation	Blocked (F15)

4. UML Diagrams

All diagrams are maintained as editable PlantUML sources in `docs/M1/diagrams/src/` and rendered to PNG and SVG. Names used in the diagrams are identical to the names in the database schema and in the API contract.

4.1. Use Case Diagram

Actors: **Guest** (not logged in), **Registered User** (inherits the guest's use cases), **NBP Web API** (external system providing rates) and **Scheduler** (timer inside the service that refreshes rate tables).

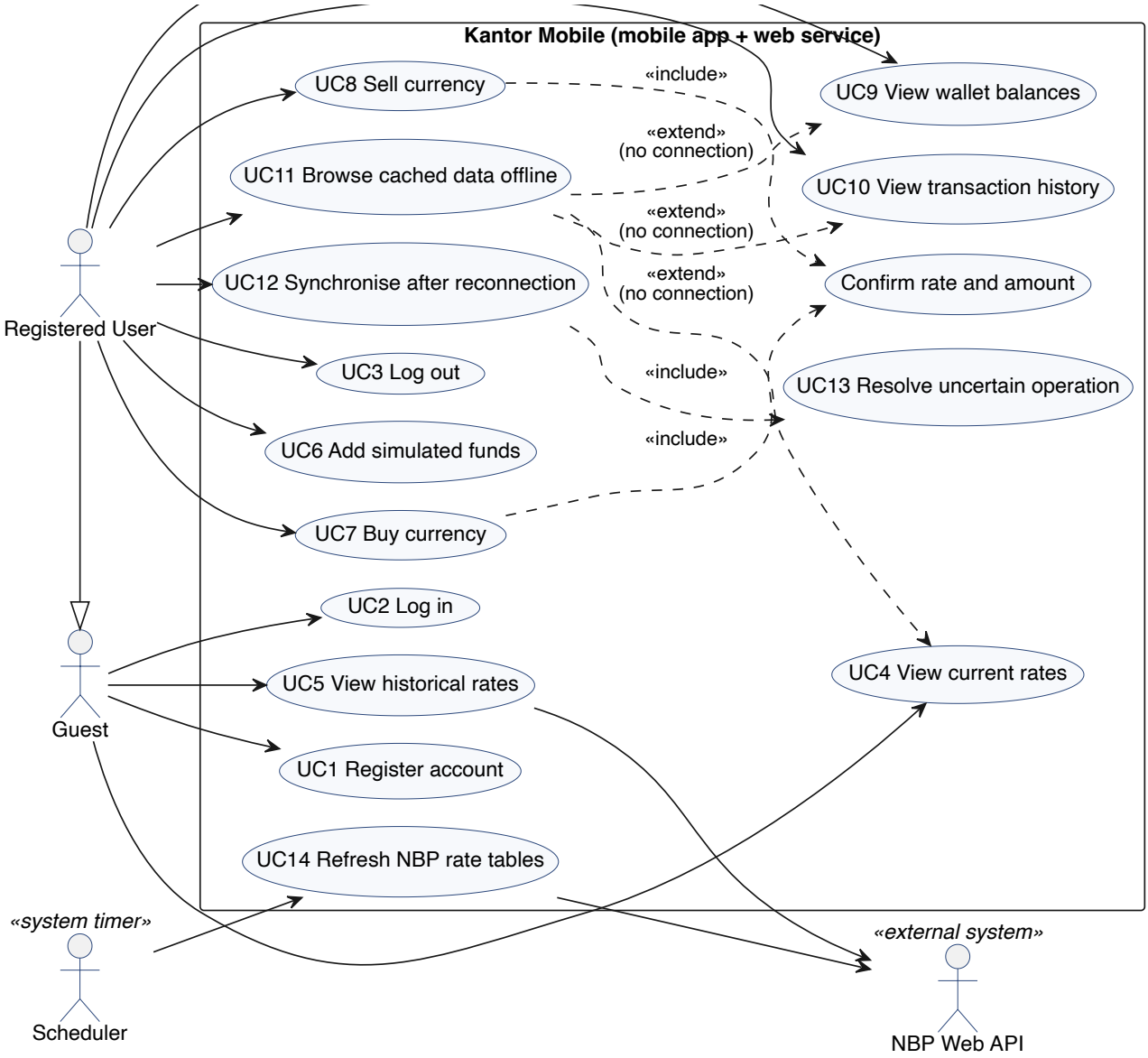


Figure 1. UML use case diagram

Table 8. Use case descriptions

ID	Use case	Main success scenario / notes	Requirements
UC1	Register account	Guest submits e-mail, password and name; service creates user and PLN wallet.	F1
UC2	Log in	Guest submits credentials; service returns a token; app stores it securely.	F2

ID	Use case	Main success scenario / notes	Requirements
UC3	Log out	Token and cached data are removed from the device.	F3
UC4	View current rates	App shows cached table immediately, then the current table from the service.	F4, F12
UC5	View historical rates	User selects currency and period; service returns stored or freshly fetched NBP data.	F5, F12
UC6	Add simulated funds	User enters PLN amount and confirms; service credits PLN wallet once.	F6, F16
UC7	Buy currency	Includes “Confirm rate and amount”; service debits PLN and credits foreign wallet atomically.	F7, F16, F19
UC8	Sell currency	Includes “Confirm rate and amount”; service debits foreign wallet and credits PLN atomically.	F8, F16, F19
UC9	View wallet balances	Authoritative balances from the service; cached snapshot offline.	F9
UC10	View transaction history	Paged list from the service; first page cached.	F10
UC11	Browse cached data offline	Extends UC4, UC9 and UC10 when there is no connection; shows freshness labels; blocks changes.	F13–F15
UC12	Synchronise after reconnection	Triggered by restored connectivity or app foreground; includes UC13 and refreshes data.	F18
UC13	Resolve uncertain operation	Asks the service for the outcome of each unconfirmed operation id and shows it.	F17
UC14	Refresh NBP rate tables	Scheduler downloads table C and stores new rows.	F12, N13

4.2. Class Diagram

The handbook requires the class model to cover both the mobile and the service side. Figure 2 shows the mobile client (TypeScript modules modelled as classes); Figure 3 shows the web service (controllers, domain services and JPA entities). Both are printed on landscape pages for legibility.

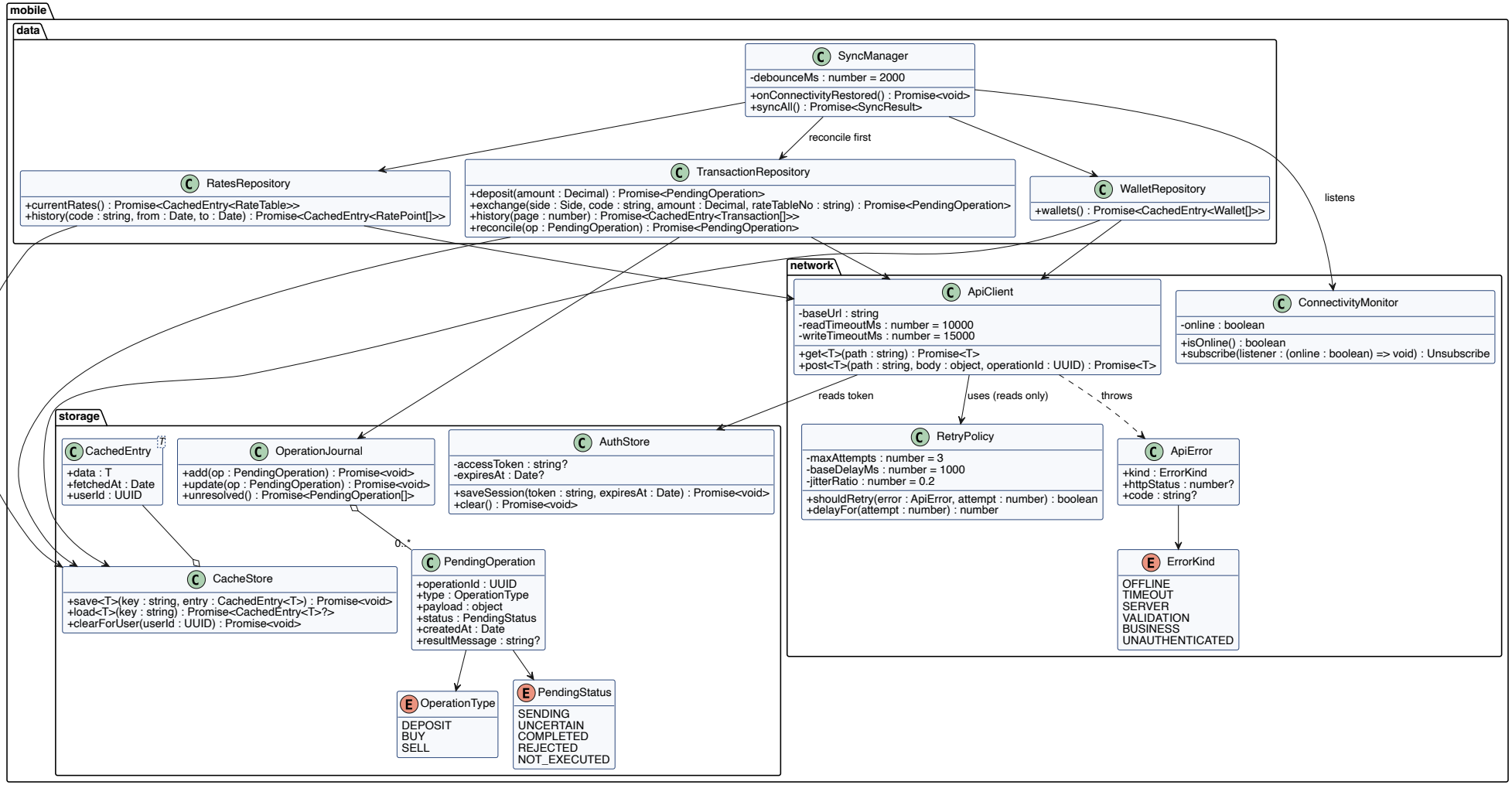


Figure 2. Class diagram — mobile client

Mobile client. `ApiClient` is the only component that talks to the network; it applies the timeouts and, for GET requests only, the `RetryPolicy`, and converts every failure into an `ApiError` with a kind (OFFLINE, TIMEOUT, SERVER, VALIDATION, BUSINESS, UNAUTHENTICATED) so the UI can show the right message. Repositories implement a cache-then-network strategy using `CacheStore`, which wraps each value in a `CachedEntry` with `fetchedAt`. `TransactionRepository` creates a `PendingOperation` with a new operation id when the user confirms an action and stores it in the persistent `OperationJournal` before the request is sent. `SyncManager` listens to `ConnectivityMonitor` and runs reconciliation and refresh after reconnection.

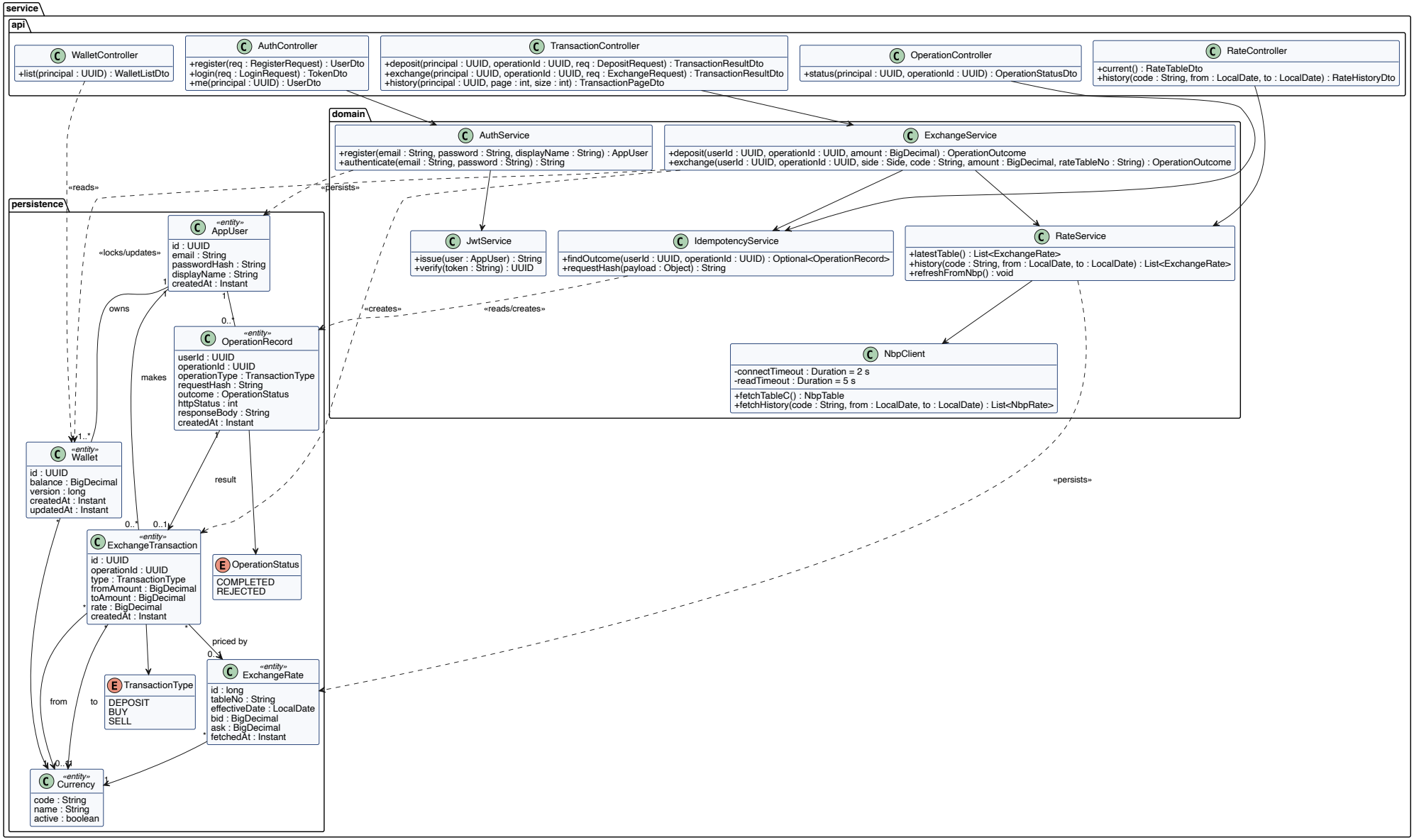


Figure 3. Class diagram — web service

Web service. Controllers translate HTTP into calls on domain services. `ExchangeService` executes deposits and exchanges inside one database transaction and uses `IdempotencyService` to look up or store the outcome of an operation id. `RateService` reads stored rates and refreshes them through `NbpClient`, which has its own short timeouts so a slow NBP API cannot block client requests. The entities correspond one-to-one to the database tables in Section 5.

4.3. Client-side state of a data-changing operation

Figure 4 complements the class diagram with the life cycle of `PendingOperation.status`. It is the core of the uncertain-outcome design: the app never moves from SENDING to a failure state on a timeout; it moves to UNCERTAIN and waits for the authoritative answer.

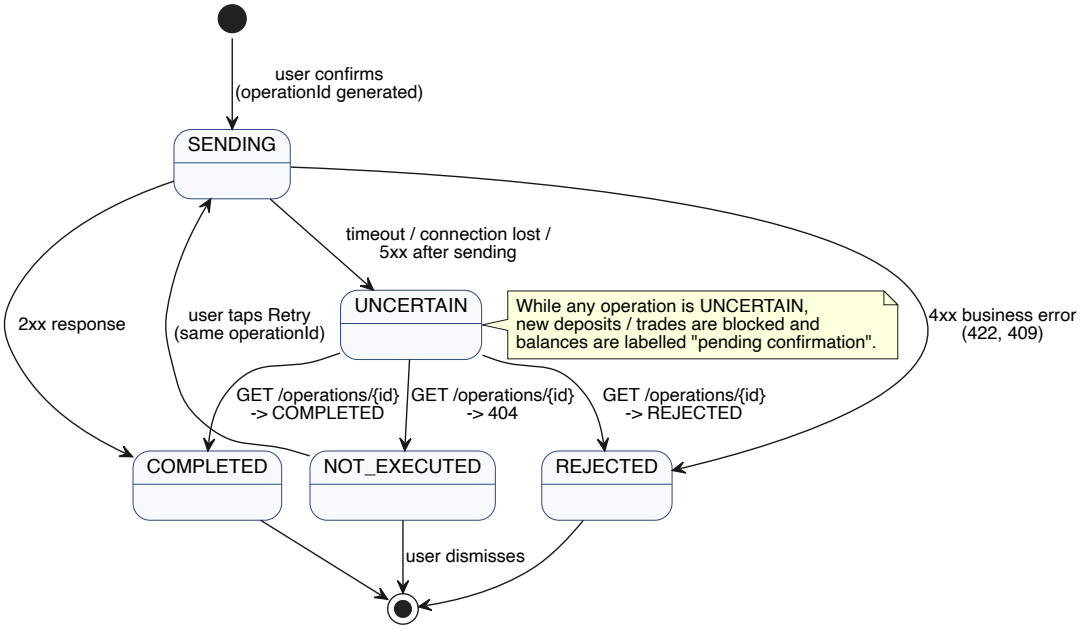


Figure 4. State machine of a data-changing operation on the device

4.4. Requirements traceability

Table 9. Requirement → use case → responsible component

Requirement	Use case	Responsible components (mobile / service / database)
F1, F2, F3	UC1–UC3	Register/Login screens, AuthStore / AuthController, AuthService, JwtService / app_user, wallet
F4, F5, F12	UC4, UC5, UC14	RatesRepository, CacheStore / RateController, RateService, NbpClient, Scheduler / exchange_rate
F6–F8, F19	UC6–UC8	Exchange & Deposit screens, TransactionRepository / TransactionController, ExchangeService / wallet, exchange_transaction
F9, F10	UC9, UC10	WalletRepository, TransactionRepository / WalletController, TransactionController / wallet, exchange_transaction
F11	all	Security filter (JWT), validation in controllers and ExchangeService / constraints in all tables
F13–F15	UC11	ConnectivityMonitor, CacheStore, offline banner, disabled actions
F16, F17	UC6–UC8, UC13	OperationJournal, ApiClient (Idempotency-Key) / IdempotencyService, OperationController / operation_record

Requirement	Use case	Responsible components (mobile / service / database)
F18	UC12	SyncManager / OperationController, WalletController, TransactionController, RateController

5. Database Design

The database is PostgreSQL 16. The schema below is the design target; it is provided as `docs/M1/database/schema.sql` and will become the first Flyway migration in week 5. Table names are singular and in snake_case; the Java entities use the same names in CamelCase.

5.1. Entity Relationship Diagram

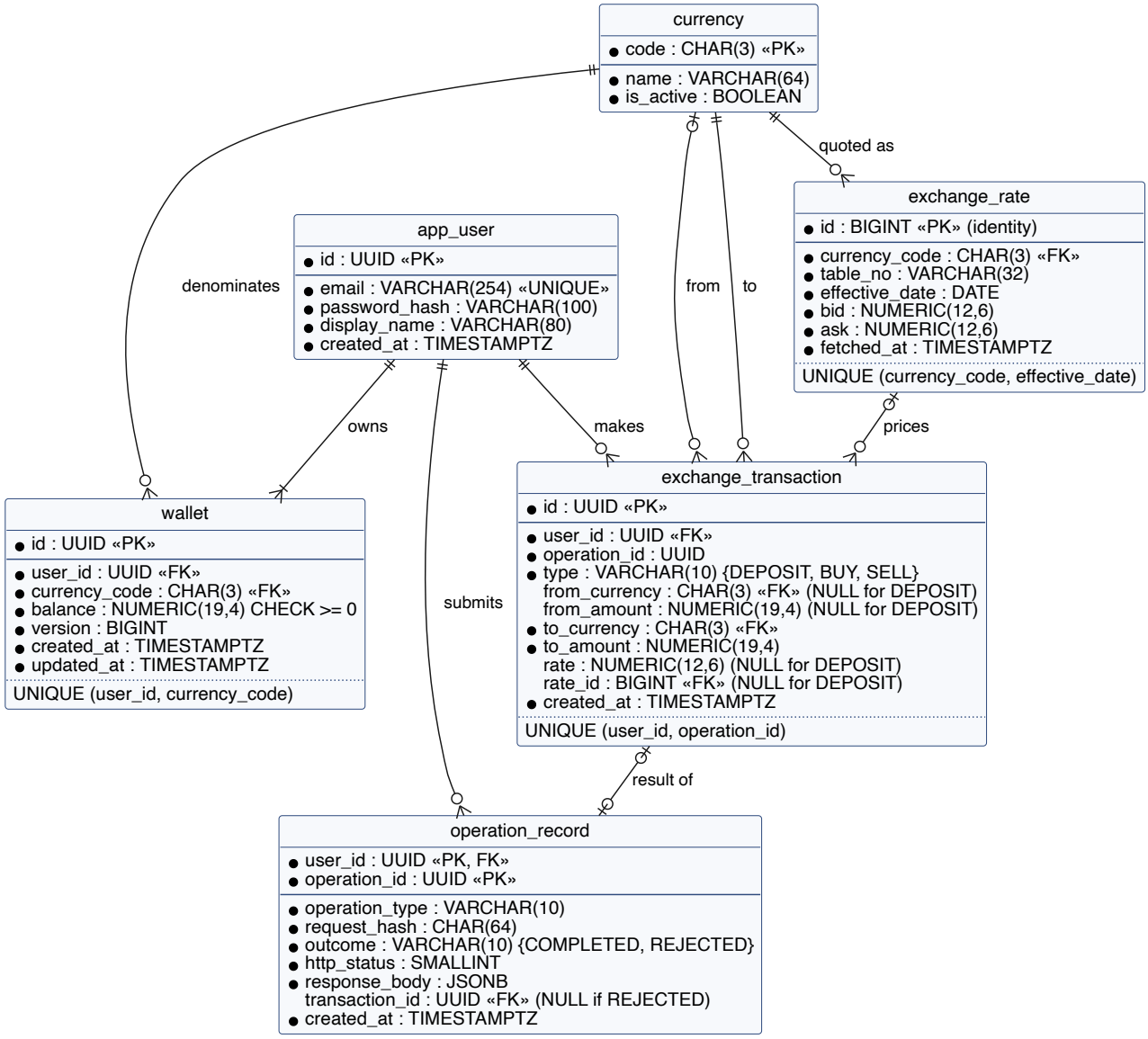


Figure 5. Entity relationship diagram (crow's-foot notation)

5.2. Entities, keys and data types

Table 10. app_user — registered users

Column	Type	Key / constraint	Description
id	UUID	PK	Generated by the service
email	VARCHAR(254)	NOT NULL, UNIQUE	Login name, stored lower-case

Column	Type	Key / constraint	Description
password_hash	VARCHAR(100)	NOT NULL	BCrypt hash; the password itself is never stored
display_name	VARCHAR(80)	NOT NULL	Name shown in the app
created_at	TIMESTAMPTZ	NOT NULL, default now()	Registration time

Table 11. currency — supported currencies (reference data)

Column	Type	Key / constraint	Description
code	CHAR(3)	PK	ISO 4217 code: PLN, USD, EUR, GBP, CHF
name	VARCHAR(64)	NOT NULL	English name
is_active	BOOLEAN	NOT NULL, default TRUE	Allows disabling a currency without deleting history

Table 12. wallet — one balance per user and currency

Column	Type	Key / constraint	Description
id	UUID	PK	
user_id	UUID	FK → app_user.id, NOT NULL	Owner
currency_code	CHAR(3)	FK → currency.code, NOT NULL	Currency of the wallet
balance	NUMERIC(19,4)	NOT NULL, CHECK (balance >= 0)	Authoritative balance
version	BIGINT	NOT NULL, default 0	Incremented on each change; exposes concurrent updates in tests
created_at, updated_at	TIMESTAMPTZ	NOT NULL	Audit times; updated_at shown in the app
(user_id, currency_code)		UNIQUE	At most one wallet per currency per user

Table 13. exchange_rate — stored NBP table C rows

Column	Type	Key / constraint	Description
id	BIGINT	PK (identity)	
currency_code	CHAR(3)	FK → currency.code	
table_no	VARCHAR(32)	NOT NULL	NBP table number, e.g. 212/C/NBP/2026
effective_date	DATE	NOT NULL	Publication date of the table
bid	NUMERIC(12,6)	NOT NULL, > 0	Price at which the user sells the currency
ask	NUMERIC(12,6)	NOT NULL, > 0, ask ≥ bid	Price at which the user buys the currency
fetched_at	TIMESTAMPTZ	NOT NULL	When the service downloaded the row
(currency_code, effective_date)		UNIQUE	One row per currency per table; makes refresh idempotent

Table 14. exchange_transaction — executed deposits, buys and sells

Column	Type	Key / constraint	Description
id	UUID	PK	
user_id	UUID	FK → app_user.id, NOT NULL	
operation_id	UUID	NOT NULL; UNIQUE with user_id	Client operation identity (second line of duplicate protection)
type	VARCHAR(10)	CHECK IN (DEPOSIT, BUY, SELL)	
from_currency	CHAR(3)	FK → currency.code, NULL for DEPOSIT	PLN for BUY, foreign for SELL
from_amount	NUMERIC(19,4)	NULL for DEPOSIT, > 0	Amount debited
to_currency	CHAR(3)	FK → currency.code, NOT NULL	PLN for DEPOSIT and SELL
to_amount	NUMERIC(19,4)	NOT NULL, > 0	Amount credited
rate	NUMERIC(12,6)	NULL for DEPOSIT	Ask (BUY) or bid (SELL) applied
rate_id	BIGINT	FK → exchange_rate.id, NULL for DEPOSIT	Exact NBP row used
created_at	TIMESTAMPTZ	NOT NULL	Execution time

Table 15. operation_record — stored outcome of each data-changing request

Column	Type	Key / constraint	Description
user_id	UUID	PK (part), FK → app_user.id	Keys are scoped per user
operation_id	UUID	PK (part)	Value of the Idempotency-Key header
operation_type	VARCHAR(10)	CHECK IN (DEPOSIT, BUY, SELL)	
request_hash	CHAR(64)	NOT NULL	SHA-256 of the canonical request body; detects reuse of a key for a different request
outcome	VARCHAR(10)	CHECK IN (COMPLETED, REJECTED)	Business outcome
http_status	SMALLINT	NOT NULL	Status of the original response (201, 409, 422)
response_body	JSONB	NOT NULL	Original response, replayed for duplicates and returned by reconciliation
transaction_id	UUID	FK → exchange_transaction.id ; NOT NULL iff COMPLETED	Link to the executed transaction
created_at	TIMESTAMPTZ	NOT NULL	Processing time

5.3. Relationships

Table 16. Relationships and foreign keys

Parent	Child	Cardinality	Foreign key / rule
app_user	wallet	1 : 1..*	wallet.user_id; PLN wallet created at registration; ON DELETE CASCADE
currency	wallet	1 : 0..*	wallet.currency_code
currency	exchange_rate	1 : 0..*	exchange_rate.currency_code
app_user	exchange_transaction	1 : 0..*	exchange_transaction.user_id
currency	exchange_transaction	1 : 0..* (twice)	from_currency (optional) and to_currency
exchange_rate	exchange_transaction	0..1 : 0..*	rate_id; NULL for deposits
app_user	operation_record	1 : 0..*	operation_record.user_id
exchange_transaction	operation_record	1 : 0..1	operation_record.transaction_id, unique when not NULL

5.4. Rationale for data types

- **UUID keys** for users, wallets and transactions can be generated without a database round trip and do not reveal how many records exist. The operation id is also a UUID because it is generated on the device, where collisions with other devices must be practically impossible.
- **NUMERIC(19,4)** for balances and amounts and **NUMERIC(12,6)** for rates store decimal values exactly; NBP publishes rates with four decimal places, so six leave headroom.
- **TIMESTAMPTZ** stores absolute instants; the app converts them to the device's time zone. **DATE** is used for the NBP effective date because NBP tables are per day.
- **JSONB** keeps the original response body for replay without designing a separate table for each response type.
- **CHECK constraints** encode the shape of each transaction type and the non-negative balance, so a programming error in the service cannot write inconsistent data.

5.5. Consistency rules enforced by the database

1. A wallet balance can never be negative (`ck_wallet_balance_non_negative`).
2. A user has at most one wallet per currency (`uq_wallet_user_currency`).
3. An operation id can produce at most one transaction per user (`uq_exchange_transaction_operation`) and at most one stored outcome (`operation_record` primary key).
4. A completed outcome always references a transaction and a rejected one never does (`ck_operation_record_link`).
5. Deposits are PLN only; buys go from PLN to a foreign currency; sells go from a foreign currency to PLN (`ck_exchange_transaction_shape`).

The DDL draft was loaded into an embedded PostgreSQL engine (PGlite) to check its syntax and that the constraints reject negative balances, duplicate operation ids, a deposit in a foreign currency, a completed outcome without a transaction and a duplicate wallet. This is a check of the design artefact only, not a test of the application.

5.6. Data stored on the mobile device

Table 17. Local storage on the device (not authoritative)

Key / store	Content	Rules
SecureStore: session	Access token, expiry, user id	Deleted on logout or on HTTP 401
AsyncStorage: rates:current	Last table C with tableNo, effectiveDate, fetchedAt	Shown offline with retrieval time; “outdated” after 24 h
AsyncStorage: rates:history: {code}	Last viewed range per currency	Replaced when a newer range is fetched
AsyncStorage: {userId}:wallets	Balances and asOf time	Replaced only by a successful server response
AsyncStorage: {userId}:transactions:0	First page of history and asOf time	Replaced only by a successful server response
AsyncStorage: {userId}:operations	OperationJournal (pending operations)	Kept across logout; resolved entries removed after 7 days

6. System Architecture

6.1. Components and interconnections

Figure 6 shows the components, the communication paths and the owner of each kind of data. The only path between the device and the service crosses the **wireless segment**, which may lose, delay or cut off messages in both directions. The mobile application never talks to the database or to the NBP API directly.

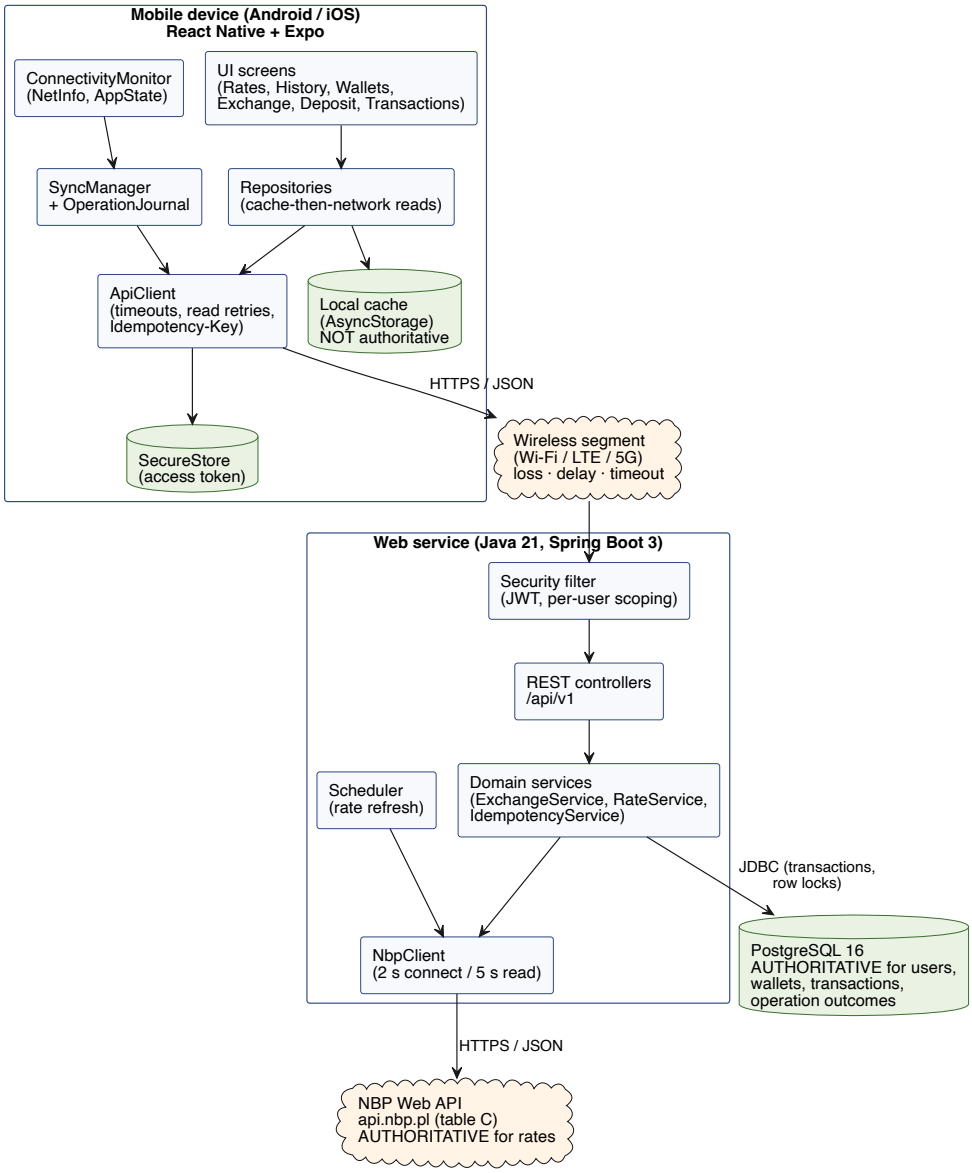


Figure 6. System architecture and data owners

Table 18. Data owners

Data	Authoritative owner	Copies
Users, credentials	Web service / PostgreSQL	Token and profile on the device
Wallet balances, transactions	Web service / PostgreSQL	Read-only snapshot in the device cache
Outcome of a data-changing operation	Web service / PostgreSQL (`operation_record`)	Pending status in the device journal until confirmed

Data	Authoritative owner	Copies
Exchange rates	NBP Web API	Stored rows in PostgreSQL; read-only cache on the device

6.2. Technology stack

The handbook offers technology examples, not mandatory technologies. Mandatory are an integrated mobile client, web service and database, Git-based development and the network-resilience behaviour. The concrete stack is a team decision.

Table 19. Mandatory elements and chosen technologies

Element	Status	Chosen technology
Mobile client	Mandatory (technology free)	React Native with Expo SDK, TypeScript; NetInfo, AsyncStorage, SecureStore
Web service	Mandatory (technology free)	Java 21, Spring Boot 3 (Web, Security, Data JPA, Validation), REST/JSON
Database	Mandatory (technology free)	PostgreSQL 16, Flyway migrations, Docker Compose for local start-up
External data	Mandatory for Project A: NBP API	NBP Web API, table C, JSON format
Version control	Mandatory: Git with history across the semester	Git, private GitHub repository
Communication	Must be documented	HTTPS (HTTP on the local development network), JSON, JWT bearer token

6.3. Data flows

Three data flows are specified: a normal read, a data-changing action and the recovery of an interrupted action. In every flow the **web service decides whether an action succeeded**; the mobile app only displays that decision.

DF-1 — Normal read: current rates

The repository shows cached data immediately (labelled with its age) and then requests fresh data. A successful response replaces the cache. On failure the cached data remains visible with an offline or “could not refresh” banner. The service answers from the database; NBP is contacted only by the scheduler, so an NBP outage never slows down this request.

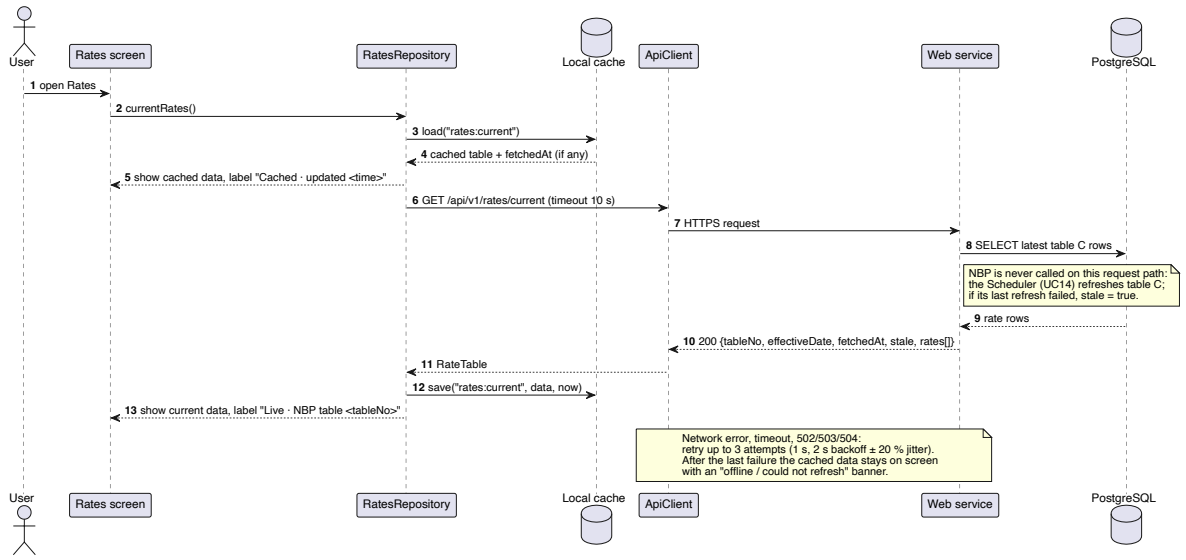


Figure 7. DF-1: reading current rates (cache-then-network)

DF-2 — Data-changing action: buy currency

When the user confirms, the app generates the operation id once, stores the pending operation in the journal and sends the request with the `Idempotency-Key` header. The service locks the user's wallets, checks whether this operation id was already processed (and if so replays the stored result), checks the rate table and the balance, and commits the wallet updates, the transaction and the operation record in one database transaction.

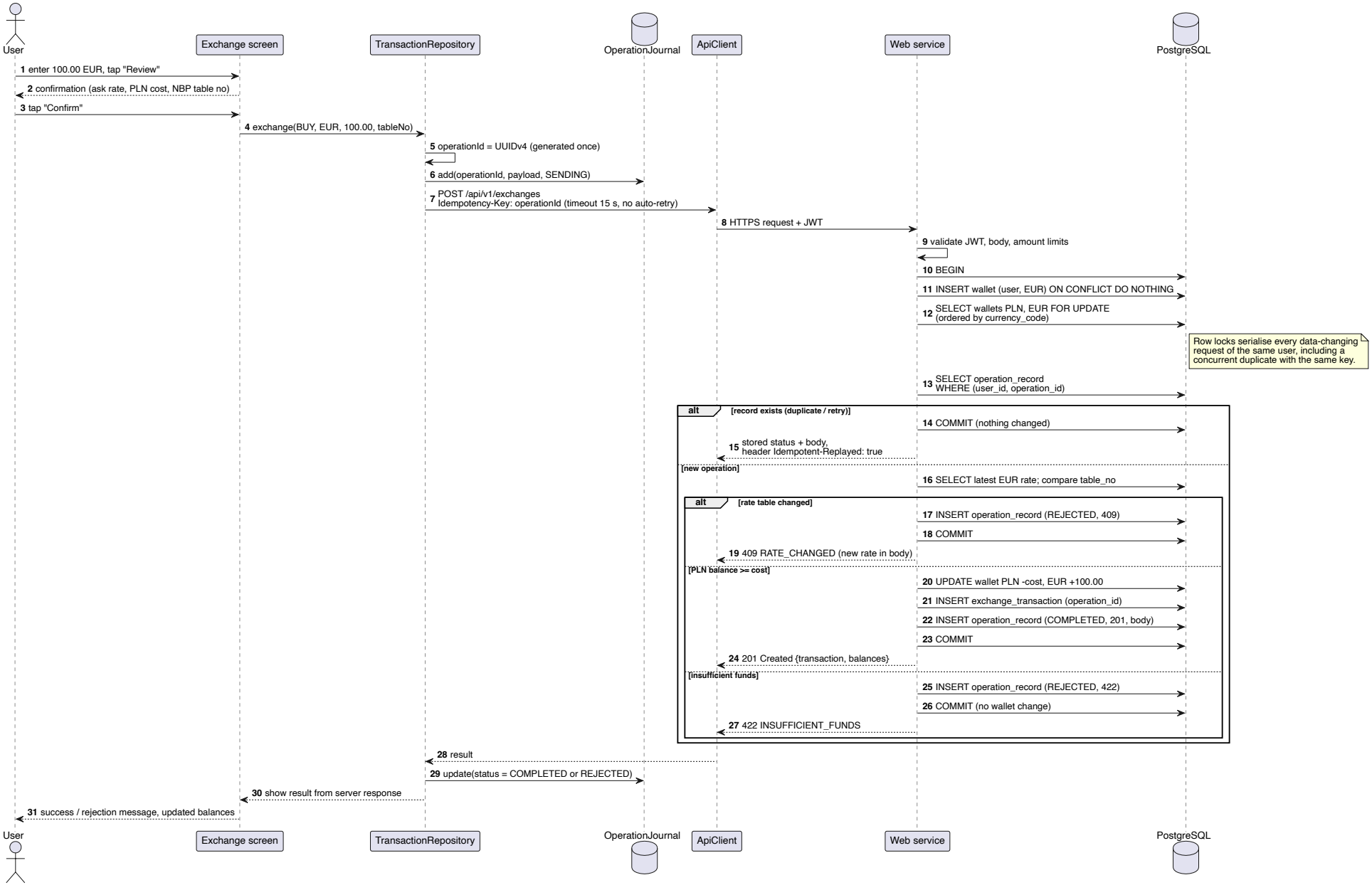


Figure 8. DF-2: buying currency with a stable operation identity

DF-3 — Recovery of an interrupted action

Figure 9 distinguishes the two cases the handbook requires: (A) the server committed but the response was lost, and (B) the request never reached the server. The app cannot distinguish them at the moment of the timeout, so it treats both as UNCERTAIN. After reconnection the reconciliation endpoint gives the authoritative answer: case A returns the stored outcome, case B returns 404, and only then may the same request be resent with the same operation id.

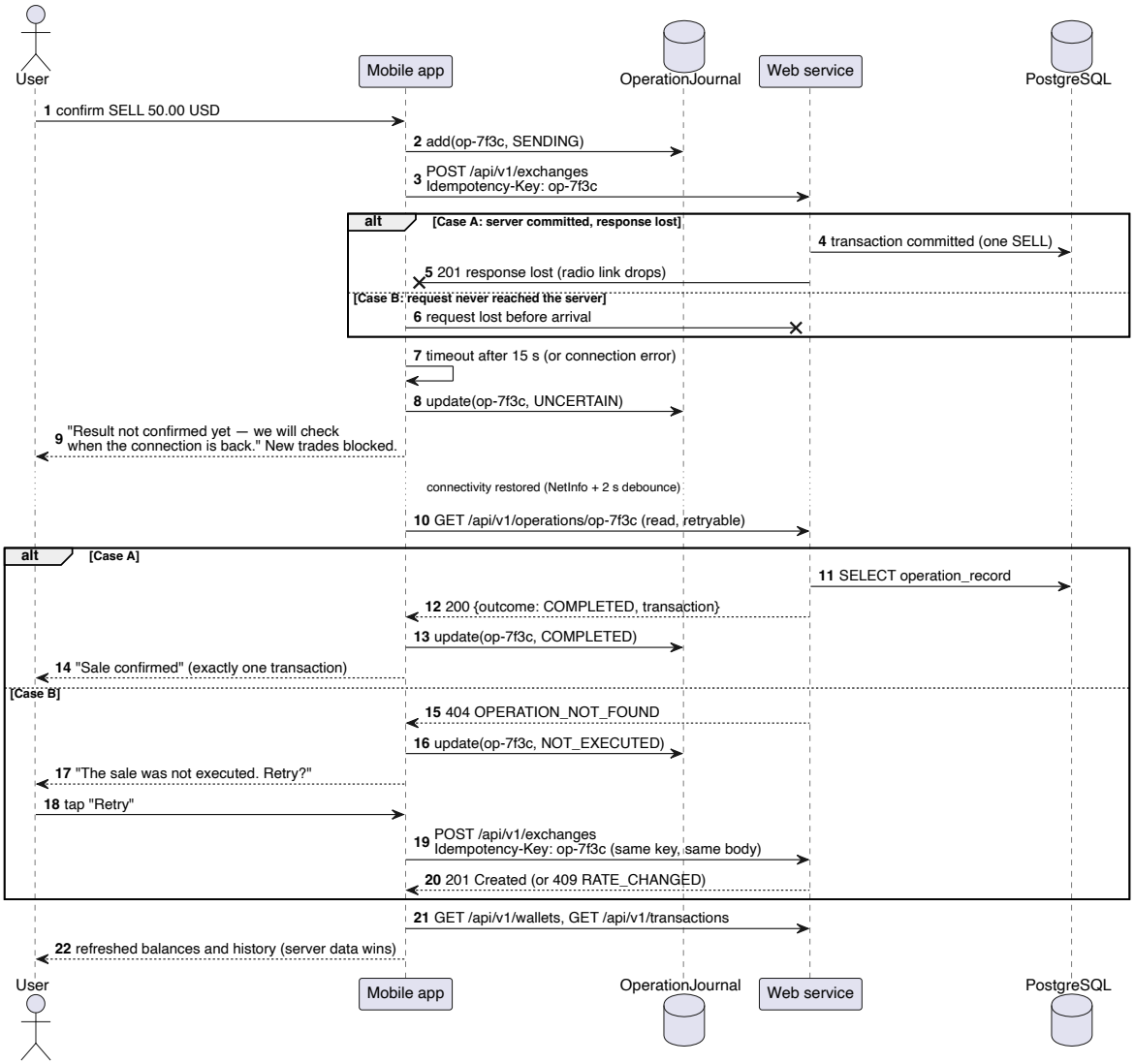


Figure 9. DF-3: recovery after a lost response

6.4. API contract

The full contract is provided as OpenAPI 3.1 in `docs/M1/api/api-contract.yaml`. Conventions: base path `/api/v1`; JSON; amounts and rates as decimal **strings**; timestamps in ISO 8601 UTC; errors as RFC 9457 Problem Details [14] with a stable `code`; protected endpoints require `Authorization: Bearer <token>`.

Table 20. Service operations

Method	Path	Auth	Idempotency-Key	Purpose and main responses
GET	/health	No	—	Liveness check after reconnection. 200
POST	/auth/register	No	—	Create account and PLN wallet. 201; 400; 409 EMAIL_TAKEN

Method	Path	Auth	Idempotency-Key	Purpose and main responses
POST	/auth/login	No	—	Obtain access token. 200; 401 INVALID_CREDENTIALS
GET	/me	Yes	—	Current user profile. 200; 401
GET	/rates/current	No	—	Latest table C with tableNo, effectiveDate, fetchedAt, stale. 200; 503 RATES_UNAVAILABLE
GET	/rates/{code}/history?from&to	No	—	Historical bid/ask, at most 93 days. 200; 400; 404; 503
GET	/wallets	Yes	—	All balances with asOf. 200; 401
POST	/deposits	Yes	Required	Simulated PLN deposit. 201 (replay: same body + Idempotent-Replayed: true); 400; 409 IDEMPOTENCY_CONFLICT
POST	/exchanges	Yes	Required	BUY or SELL against PLN. 201; 400; 409 RATE_CHANGED / IDEMPOTENCY_CONFLICT; 422 INSUFFICIENT_FUNDS; 503
GET	/transactions?page&size	Yes	—	History, newest first. 200; 401
GET	/operations/{operationId}	Yes	—	Reconciliation: stored outcome (200) or OPERATION_NOT_FOUND (404)

Representative request and response — buy 100.00 EUR:

```
POST /api/v1/exchanges
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Idempotency-Key: 7f3c2a90-1b4e-4d8f-9c11-5e6a7b8c9d0e
Content-Type: application/json

{ "side": "BUY", "currency": "EUR", "amount": "100.00",
  "rateTableNo": "212/C/NBP/2026" }

HTTP/1.1 201 Created
Content-Type: application/json

{ "operationId": "7f3c2a90-1b4e-4d8f-9c11-5e6a7b8c9d0e",
  "transaction": { "id": "0d6e2f4a-8b1c-4e7d-a2f3-6b5c4d3e2f1a",
    "type": "BUY", "fromCurrency": "PLN", "fromAmount": "429.21",
    "toCurrency": "EUR", "toAmount": "100.00", "rate": "4.2921",
    "rateTableNo": "212/C/NBP/2026", "createdAt": "2026-11-03T09:14:55Z" },
  "balances": [ { "currency": "PLN", "balance": "570.79" },
                { "currency": "EUR", "balance": "100.00" } ] }
```

Listing 1. Successful exchange (100.00 × 4.2921 = 429.21 PLN; 1,000.00 − 429.21 = 570.79)

The same request repeated with the same key (for example after the first response was lost) returns the identical status and body with the additional header `Idempotent-Replayed: true`; no second transaction is created. The same key with a different body returns `409 IDEMPOTENCY_CONFLICT`.

```
GET /api/v1/operations/7f3c2a90-1b4e-4d8f-9c11-5e6a7b8c9d0e
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

HTTP/1.1 200 OK
{ "operationId": "7f3c2a90-1b4e-4d8f-9c11-5e6a7b8c9d0e", "type": "BUY",
  "outcome": "COMPLETED", "httpStatus": 201,
  "processedAt": "2026-11-03T09:14:55Z", "response": { ...original body... } }

HTTP/1.1 404 Not Found                        (request never processed)
Content-Type: application/problem+json
{ "type": "about:blank", "title": "Not Found", "status": 404,
  "code": "OPERATION_NOT_FOUND" }
```

Listing 2. Reconciliation of an uncertain operation

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json
{ "type": "about:blank", "title": "Unprocessable Content", "status": 422,
  "code": "INSUFFICIENT_FUNDS",
  "detail": "PLN balance 120.00 is lower than the required 429.21.",
  "operationId": "7f3c2a90-1b4e-4d8f-9c11-5e6a7b8c9d0e" }
```

Listing 3. Business rejection (stored and replayed for the same key)

Table 21. Error codes

Code	HTTP	Meaning and client reaction
VALIDATION_ERROR	400	Invalid input; field errors shown next to the fields. Not stored; nothing changed.
UNAUTHENTICATED / INVALID_CREDENTIALS	401	Token missing or expired / wrong password; app opens the login screen.
CURRENCY_NOT_SUPPORTED, OPERATION_NOT_FOUND	404	Unknown resource; for operations: request never processed (Section 6.5.5).
EMAIL_TAKEN	409	Registration with an existing e-mail; app offers login.
RATE_CHANGED	409	NBP table changed since review; new price shown for a fresh confirmation with a new operation id.
IDEMPOTENCY_CONFLICT	409	Key reused for a different request; indicates a client defect; nothing changed.
INSUFFICIENT_FUNDS	422	Balance too low; nothing changed.
RATES_UNAVAILABLE	503	No sufficiently fresh rate table; reads may be retried later.

6.5. Network-failure handling design

6.5.1. Connectivity detection

The app combines three signals: platform network events from NetInfo (`isConnected` and `isInternetReachable`), the result of every API request (a network error or timeout immediately switches the UI to the offline state) and the app returning to the foreground. Connectivity is considered **restored** when the platform reports a connection and `GET /health` succeeds; the state must be stable for 2 s before synchronisation starts, which prevents repeated synchronisation on a flapping link.

6.5.2. Timeout policy

Table 22. Timeouts (all values are design targets to be verified at M3)

Where	Value	Reason
Client → service, read (GET)	10 s	Far above the 500 ms service target (N1), leaving room for slow mobile links and TLS set-up, but short enough that the user is not left waiting; cached data is visible meanwhile.
Client → service, data-changing (POST)	15 s	Longer than the read timeout because a premature timeout on a write creates an uncertain outcome; it is set above the server-side limits below so that the server normally finishes or aborts before the client gives up.
Service → PostgreSQL	lock_timeout 5 s, statement_timeout 10 s	A blocked or slow transaction is rolled back by the server before the client's 15 s timeout.
Service → NBP API	connect 2 s, read 5 s, 1 retry	NBP is called only by the scheduler and for missing history ranges; a slow NBP response must not hold client requests.
UI feedback	300 ms / 3 s	Spinner after 300 ms; "Still working…" after 3 s; offline banner on failure.

6.5.3. Retry policy

- **Reads** (all GET requests) are retried automatically: at most 3 attempts in total, waiting 1 s and then 2 s with ±20 % random jitter to avoid synchronised retries from many devices [13]. Only network errors, timeouts and HTTP 502/503/504 are retried; 4xx responses and other 5xx responses are not. Retrying stops at once if the device reports that it is offline.
- **Data-changing requests** (deposit, buy, sell) are **never re-sent automatically**. After a timeout, connection error or 5xx response the operation becomes UNCERTAIN and is resolved through reconciliation (6.5.5). A manual "Retry" after a 404 from reconciliation resends the **same body with the same operation id**, so even a mistaken resend cannot have a second effect.
- Registration is not retried automatically; if a lost response is followed by a manual retry, EMAIL_TAKEN tells the app that the account already exists and the app offers login.

6.5.4. Stable operation identity and duplicate protection

The operation id is a UUID v4 generated on the device **at the moment the user confirms** an action — not when the request is sent — and it is written to the persistent OperationJournal before the first transmission. Every transmission of that action, including after an app restart, uses the same id in the `Idempotency-Key` header. A new id is generated only for a new user decision (for example after RATE_CHANGED the user confirms a new price). On the service, `ExchangeService` processes a request as follows, in a single database transaction:

1. Ensure the target wallet exists (`INSERT … ON CONFLICT DO NOTHING`).
2. Lock all wallets involved with `SELECT … FOR UPDATE`, always in currency-code order to avoid deadlocks. These row locks serialise all data-changing requests of the same user, including two copies of the same request arriving at the same time.
3. Look up `operation_record` for (user, operation id). If it exists and the request hash matches, return the stored status and body with `Idempotent-Replayed: true` and change nothing; if the hash differs, return 409 IDEMPOTENCY_CONFLICT.
4. Otherwise validate the business rules (rate table unchanged, sufficient balance), update the balances, insert the transaction and insert the operation record with the response body, then commit.

5. If a business rule fails, insert only the operation record with outcome REJECTED (409 or 422) and commit, so a retry of the same operation receives the same rejection.

The unique constraint on (user_id, operation_id) in `exchange_transaction` and the primary key of `operation_record` are a second line of defence: even if the service logic were wrong, the database would refuse a second transaction for the same operation id.

6.5.5. Recovery when a server response is lost

A timeout does not mean failure. The app therefore uses the UNCERTAIN state (Figure 4) and never shows "failed" for a data-changing request whose outcome it does not know. While an operation is UNCERTAIN the affected balances are labelled "pending confirmation" and new deposits and trades are blocked, so the user cannot unknowingly spend money twice. After reconnection (or after the next login if the token expired) `SyncManager` calls `GET /operations/{id}` for every unresolved operation:

Table 23. Reconciliation outcomes

Reconciliation result	Meaning	App behaviour
200, outcome COMPLETED	Server committed; only the response was lost	Show the stored success result; refresh balances.
200, outcome REJECTED	Server processed and rejected (e.g. insufficient funds)	Show the stored rejection; nothing changed.
404 OPERATION_NOT_FOUND	Server never received the request, or rolled it back	Mark NOT_EXECUTED; offer "Retry" with the same operation id, or dismiss.
Network error / timeout	Still unknown	Keep UNCERTAIN; retry reconciliation as a read (N3).

6.5.6. Offline mode and cache

Decision: data-changing actions are blocked while offline; nothing is queued. Queuing a buy or sell would mean executing it later at an unknown price and against a balance that may have changed, and the app would have to show a "pending" order that might fail much later. Blocking keeps the semantics simple and honest: the server state is unchanged and the user is told why the action is unavailable. Available offline: login state, current rates, the last viewed rate history, balances and the first page of transactions, each with the label "Cached · updated <time>" (rates additionally show the NBP table number and effective date). Unavailable offline: registration, login, deposit, buy, sell, older history pages and history ranges not viewed before.

6.5.7. Reconnection and synchronisation

1. Confirm reachability (`GET /health`) after 2 s of stable connectivity.
2. Resolve every UNCERTAIN operation in the journal (6.5.5) — first, because its result changes balances.
3. Reload wallets and the first page of transactions; replace the cached copies.
4. Reload current rates; reload the visible history range.
5. Show "Syncing…" during the process and "Up to date · <time>" at the end; on failure keep the cached data and the offline banner.

Rule: confirmed server data always replaces local data; local data never overwrites server data. Because offline changes are blocked, there are no local modifications that could conflict with the server — conflicts are reduced to stale reads, which are resolved by refreshing.

6.5.8. Consistency under concurrent requests

Two requests that change the same balance (for example two sells of the whole EUR balance from two devices) are serialised by the wallet row locks. The second request sees the balance after the first one committed and is rejected with INSUFFICIENT_FUNDS if the balance is no longer sufficient; the `CHECK (balance >= 0)` constraint guarantees that no interleaving can produce a negative balance. Deposits, buys and sells are each a single transaction at READ COMMITTED isolation with explicit row locks, which is sufficient because all reads that influence a decision are made on locked rows.

6.5.9. Failure of the external data source

If NBP is unreachable, the scheduler keeps the previous table and marks the next `/rates/current` response with `stale: true`; the app shows “NBP update delayed” next to the table date. Trading continues with the latest stored table as long as it is not older than 5 calendar days (covering weekends and public holidays, when NBP publishes no tables); after that the service rejects trades with RATES_UNAVAILABLE. History ranges that are not stored and cannot be downloaded return 503 and the app shows the cached history if any.

6.6. Test environment and fault injection (planned)

- **No connection:** airplane mode on the Android emulator or device; disabling Wi-Fi on the development laptop.
- **Slow response and timeout:** a fault-injection filter in the service, enabled only in the `dev` Spring profile, that delays a selected endpoint by a configured time (e.g. 4 s and 20 s).
- **Response lost after commit:** the same dev-only filter can drop the connection after the transaction has committed, reproducing case A of DF-3; turning off the network immediately after tapping “Confirm” reproduces case B.
- **Duplicate and concurrent requests:** replaying a recorded request with the same `Idempotency-Key` using curl; sending two different sell requests in parallel with a small script.
- **Evidence:** screenshots or screen recordings of the app, service logs with operation ids, and SQL queries showing the resulting rows in `wallet`, `exchange_transaction` and `operation_record`.

6.7. Network-failure scenario and test matrix

The matrix lists the expected behaviour that will be tested at M3. The column “Observed / evidence” is intentionally not filled at M1; it will contain only results that were actually reproduced, with a reference to the evidence (screenshot, log extract or SQL query).

Table 24. Network-failure scenarios (expected results; observed results to be recorded at M3)

ID	Scenario	Expected result in the app	Expected server state	Observed / evidence
NS-01	Normal connection, read rates and buy	Data shown as “Live”; success with new balances	One transaction, one operation record (COMPLETED)	Not executed yet (M3)
NS-02	No connection before a read	Cached data with retrieval time and offline banner	No request received	Not executed yet (M3)
NS-03	No connection before a data-changing request	Buy/Sell/Deposit disabled with explanation; nothing sent	Unchanged	Not executed yet (M3)
NS-04	App started without connection	Last data loaded from device storage with freshness labels	No request received	Not executed yet (M3)

ID	Scenario	Expected result in the app	Expected server state	Observed / evidence
NS-05	Connection lost after sending a buy; server committed (response lost)	UNCERTAIN, trades blocked; after reconnection COMPLETED shown once	Exactly one transaction for the operation id	Not executed yet (M3)
NS-06	Connection lost before the buy reached the server	UNCERTAIN; reconciliation 404 → NOT_EXECUTED; manual retry with same id succeeds once	No transaction until retry; then exactly one	Not executed yet (M3)
NS-07	Read timeout (response delayed > 10 s)	3 attempts with backoff, then cached data and “could not refresh”	Requests received, no state change	Not executed yet (M3)
NS-08	Write timeout (response delayed > 15 s)	UNCERTAIN; reconciliation returns the real outcome	At most one transaction	Not executed yet (M3)
NS-09	Slow response below timeout (4 s)	Spinner, “Still working…” after 3 s, then normal result	One transaction	Not executed yet (M3)
NS-10	Repeated request with the same identity (double tap, replay)	Same result; second response marked as replay	Exactly one transaction; replay header present	Not executed yet (M3)
NS-11	Same identity, different request body	409 IDEMPOTENCY_CONFLICT shown as error	Unchanged	Not executed yet (M3)
NS-12	Two concurrent sells exceeding the balance	One success, one INSUFFICIENT_FUNDS	Balance ≥ 0; one transaction	Not executed yet (M3)
NS-13	Service error 5xx on a write	UNCERTAIN, then reconciliation	Rolled back (404) or committed once	Not executed yet (M3)
NS-14	NBP API unavailable	Rates shown with “NBP update delayed”; trading allowed within 5 days	Last table kept; stale = true	Not executed yet (M3)
NS-15	Rate table changes between review and confirm	New price shown; new confirmation required	REJECTED record (409); no transaction	Not executed yet (M3)
NS-16	Connectivity restored	Uncertain operations resolved first, then data refreshed; “Up to date” shown	Server data unchanged by synchronisation	Not executed yet (M3)
NS-17	Token expired while offline with an uncertain operation	Login required; reconciliation runs after login	Unchanged	Not executed yet (M3)

7. Project Implementation Plan

The plan follows the weekly structure of the laboratory handbook. Deadlines are expressed as teaching-week checkpoints; the calendar dates are those published in Moodle.

Table 25. Development stages

Stage	Description	Deadline	Responsible Person
1	Requirements analysis, network scenarios, topic and technology decision	Weeks 1–2	Both
2	Use case, class and data-flow models	Week 3	Both (Duygu: mobile classes; Hasan: service classes)
3	ERD, schema, API contract, failure design, M1 report and repository set-up	After Week 4 — M1	Hasan: ERD, schema, API; Duygu: screens, client design, review; both: report
4	Service and database foundation: Spring Boot project, Flyway migration, Docker Compose, one read and one write path with validation	Week 5	Hasan
5	Mobile client foundation: Expo project, navigation, ApiClient, loading/success/error states, first screens connected over HTTP	Week 6	Duygu
6	Registration, login, JWT authorisation, secure token storage	Week 7	Hasan (service), Duygu (screens, SecureStore)
7	NBP integration, rate storage and scheduler; rates and history screens; setup notes	After Week 8 — M2	Hasan (NbpClient, RateService), Duygu (rates and chart screens); both: M2 notes
8	Deposit, buy and sell with transactions, row locks and constraints; concurrency test	Week 9	Hasan (service), Duygu (exchange and deposit screens)
9	Timeouts, retry policy, Idempotency-Key, operation records, reconciliation endpoint, dev fault-injection filter	Week 10	Hasan (service), Duygu (client policies, journal)
10	Local cache, freshness labels, offline banner, blocking of offline actions	Week 11	Duygu
11	Reconnection detection, synchronisation and uncertain-operation resolution	Week 12	Duygu (SyncManager), Hasan (endpoint support, logs)
12	Execution of the network-failure matrix, defect fixing, evidence collection	After Week 13 — M3	Both
13	Integration and acceptance review, README completion, short user manual	Week 14	Both (Duygu: user manual; Hasan: README and setup)
14	Final report, presentation and live demonstration including an interrupted request	After Week 15 — M4	Both

7.1. Team responsibilities

Table 26. Division of work

Area	Duygu Toplu	Hasan Mert Yilmaz
Primary	Mobile application (React Native, Expo): UI/UX, screens and navigation, API integration on the client, offline interface, client-side cache, network state and error messages, mobile testing	Web service (Java, Spring Boot): REST API, PostgreSQL schema and migrations, authentication and authorisation, NBP integration, transaction validation, data consistency, duplicate-request protection, backend testing
Documentation	User manual; screen flows; client-side sections of reports; presentation design	ERD and schema; API contract; service-side sections of reports; README and setup instructions
Shared	Requirements, architecture, UML, network-failure design, integration, test execution, final report, presentation and demonstration	

The split follows the client/service boundary so that both members can work in parallel against the API contract. Each member reviews the other's pull requests, which keeps both familiar with the whole system for the final presentation.

7.2. Repository and working practice

- Private GitHub repository with folders `mobile/`, `backend/` and `docs/`; the supervisor receives read access.
- Short-lived feature branches (`feature/<area>`), merged into `main` through pull requests reviewed by the other team member.
- Small commits with descriptive messages throughout the semester; no secrets in the repository (`.env` files are ignored, an `.env.example` documents the required variables).
- Each milestone is marked with a Git tag (`m1`, `m2`, `m3`, `m4`); the Moodle submission names the tag and the commit hash.
- The README is updated at each milestone with the set-up steps, the configuration and the procedure to reproduce the network-failure demonstration.

8. Conclusions and Possible Extensions

The design defines a three-tier system in which the web service and PostgreSQL are the single authority for balances, transactions and the outcomes of operations, while the mobile application is designed for unreliable connectivity: it never waits indefinitely, retries only safe reads, shows cached data with an explicit age, blocks financial actions while offline and resolves uncertain outcomes through a dedicated reconciliation endpoint. Duplicate execution is prevented by a client-generated operation identity that is stored with the result in the same database transaction as the balance change, backed by database constraints. The requirements, diagrams, schema, API contract and failure scenarios use the same names and can be traced to each other, which provides a consistent basis for the prototype (M2) and the network-resilience work (M3).

Known limitations of the design: only four foreign currencies and exchanges against PLN; prices are NBP reference prices published once per business day rather than live market prices; the access token is not refreshed automatically (the user logs in again after 24 h); offline modifications are not queued.

Possible extensions after the project:

- Queued offline intents with an explicit expiry and a server-side price limit (for example “buy if ask ≤ 4.30”).
- Refresh tokens and biometric unlock of the session.
- All currencies of NBP table C, and exchanges between two foreign currencies via PLN.
- Rate alerts with push notifications and a home-screen widget.
- Live synchronisation of several devices of the same user (Server-Sent Events or WebSockets).
- Clean-up job for old operation records with a documented retention period.
- An administrator dashboard for supported currencies and service health.
- Cloud deployment with HTTPS, monitoring and automated integration tests in CI.
- Polish and Turkish user-interface languages.

9. Sources

[1] Kacprowicz, M. (2026). *Laboratory project handbook | 15 weeks*. Network Issues in Mobile Systems 2026–2027, VIZJA University, MoodleTi (course material).

[2] *Project_Documentation_Template_EN.docx* — Project documentation template, Part 1: Conceptual Design. Network Issues in Mobile Systems (course material).

[3] Narodowy Bank Polski. *NBP Web API — exchange rates and gold prices*. https://api.nbp.pl/en.html (accessed 1 November 2026).

[4] Fielding, R., Nottingham, M., Reschke, J. (2022). *RFC 9110: HTTP Semantics*. IETF. https://www.rfc-editor.org/rfc/rfc9110

[5] Jena, J., Dalal, S. *The Idempotency-Key HTTP Header Field*. IETF HTTPAPI Working Group, Internet-Draft draft-ietf-httpapi-idempotency-key-header. https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/

[6] Jones, M., Bradley, J., Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. IETF. https://www.rfc-editor.org/rfc/rfc7519

[7] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation* — Numeric Types; Explicit Locking; Constraints. https://www.postgresql.org/docs/16/

[8] VMware Tanzu / Spring. *Spring Boot Reference Documentation* and *Spring Security Reference*. https://docs.spring.io/

[9] Expo. *Expo Documentation* (SDK, SecureStore, AsyncStorage, NetInfo). https://docs.expo.dev/

[10] Object Management Group (2017). *Unified Modeling Language (UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

[11] Nielsen, J. (1993). *Usability Engineering*, chapter 5 (response time limits). Academic Press.

[12] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

[13] Brooker, M. *Timeouts, retries, and backoff with jitter*. Amazon Builders' Library. https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

[14] Nottingham, M., Wilde, E., Dalal, S. (2023). *RFC 9457: Problem Details for HTTP APIs*. IETF. https://www.rfc-editor.org/rfc/rfc9457

[15] OpenAPI Initiative (2021). *OpenAPI Specification v3.1.0*. https://spec.openapis.org/oas/v3.1.0

[16] PlantUML. *PlantUML Language Reference Guide*. https://plantuml.com/

Appendix A. M1 submission contents

Table 27. Files delivered for M1 (paths relative to the repository root)

Path	Content
docs/M1/Group7_M1_Analysis_and_Design.pdf	This report (PDF)
docs/M1/Group7_M1_Analysis_and_Design.docx	Editable version of this report
docs/M1/diagrams/src/*.puml	Editable sources of all diagrams (PlantUML)
docs/M1/diagrams/png/, docs/M1/diagrams/svg/	Rendered diagrams
docs/M1/database/schema.sql	PostgreSQL DDL draft of the ERD
docs/M1/api/api-contract.yaml	API contract outline (OpenAPI 3.1)
docs/build/	Script that generates the report from a single source
README.md	Project overview, technology choices, milestone status